

Autoencoder 기반 로봇 고장진단

2025.05.14

1. 프로젝트 구조

- Autoencoder 기반 로봇 고장 진단 프로그램은 Docker 환경으로 패키징 되어 있음
- 반드시 Docker를 사용할 필요는 없으며, src/ 폴더 내 Python 코드 만으로도 로컬에서 실행할 수 있음

docker_autoenc/	# 프로그램 최상위 폴더
├── Dockerfile	# Docker 환경 설정 파일(python, pytorch, 필요한 라이브러리 자동 설치)
├── models/	# 학습된 Autoencoder 모델 파일(.pth) 보관
│ └── autoencoder.pth	# - 기본 파일 : autoencoder.pth
├── requirements.txt	# 실행에 필요한 Python 패키지 목록(torch, torchvision, numpy 등)
└── src/	# 실제 소스 코드 및 데이터
├── check_faulty_data.py	# 고장 데이터 판별 스크립트
├── check_faulty_data2.py	# (확장판) 고장 판별 스크립트 (옵션 추가, 분석 고도화)
├── check_learning_data.py	# 학습 데이터 확인 스크립트
├── train_autoenc.py	# Autoencoder 학습 스크립트
├── models/	# (비워둠) 소스코드 내 모델 디렉토리
└── wafer_data/	# 분석 대상 데이터(.txt, .csv)

2. Docker 환경 구성

- Docker 환경 사용 안할 시, 건너뛰어도 무방함
- 코드 수정 되었을 경우 Docker 이미지 빌드 해주어야 함

```
cd ~/docker_autoenc  
sudo docker build -t autoenc_pytorch .
```

- Docker 컨테이너 실행(with 모델 & 데이터)

```
sudo docker run --gpus all -it ₩  
-v $(pwd)/models:/workspace/src/models ₩  
-v $(pwd)/src/wafer_data:/workspace/src/wafer_data ₩  
autoenc_pytorch /bin/bash
```

GPU 자원 사용시 -gpus all 옵션 필수

3. 주요 스크립트 사용법

◆ 고장 데이터 판별 `check_faulty_data.py`

- `check_faulty_data.py` 는 **Autoencoder** 모델을 이용해 `wafer_data` 내 진동 데이터를 판별하고, 재구성 오류를 기준으로 고장 여부를 판단하는 스크립트

◆ 실행방법

✓ 기본 사용법(모델만 마운트)

- `Wafer_data`는 Docker 이미지에 이미 포함되어 있다고 가정할 때

```
sudo docker run --rm -it ₩  
-v ~/docker_autoenc/models:/workspace/src/models ₩  
autoenc-docker ₩  
python /workspace/src/check_faulty_data.py
```

3. 주요 스크립트 사용법

◆ 고장 데이터 판별 `check_faulty_data.py`

- `check_faulty_data.py` 는 **Autoencoder** 모델을 이용해 `wafer_data` 내 진동 데이터를 판별하고, 재구성 오류를 기준으로 고장 여부를 판단하는 스크립트

◆ 실행방법

✓ 데이터 추가 후 실행(데이터도 마운트)

- `wafer_data`에 새로운 데이터를 추가 했을 때 (로컬 → 컨테이너로 마운트)

```
sudo docker run --rm -it ₩  
-v ~/docker_autoenc/src/wafer_data:/workspace/src/wafer_data ₩  
-v ~/docker_autoenc/models:/workspace/src/models ₩  
autoenc-docker ₩  
python /workspace/src/check_faulty_data.py
```

3. 주요 스크립트 사용법

◆ 고장 데이터 판별 `check_faulty_data.py`

- `check_faulty_data.py` 는 **Autoencoder** 모델을 이용해 `wafer_data` 내 진동 데이터를 판별하고, 재구성 오류를 기준으로 고장 여부를 판단하는 스크립트

◆ 코드 동작 방식

단계	설명
1. 모델 로드	<code>models/autoencoder.pth</code> 파일을 불러와 Autoencoder 네트워크에 로드
2. 데이터 로딩	<code>wafer_data/</code> 디렉토리 내 지정된 파일 리스트를 순차적으로 로딩
3. 전처리	각 파일에서 Roll, Pitch, Yaw, AccX, AccY, AccZ 데이터를 추출 및 전처리
4. 재구성 오류 계산	Autoencoder로 재구성 후, 원본과의 MSE(Mean Squared Error) 기반 오류 계산
5. 고장 여부 판별	평균 재구성 오류가 <code>THRESHOLD (56250000)</code> 이상이면 고장으로 판단

#로봇 자세, 모션 등이 새로워질 경우, 고장 진단 판단 성능 향상을 위해 재구성 오류 Threshold 값 튜닝 필요

3. 주요 스크립트 사용법

◆ 고장 데이터 판별 `check_faulty_data.py`

- `check_faulty_data.py` 는 **Autoencoder** 모델을 이용해 `wafer_data` 내 진동 데이터를 판별하고, 재구성 오류를 기준으로 고장 여부를 판단하는 스크립트

◆ 출력 예시

파일: 260-1.txt

- 평균 재구성 오류: 78912345.123456

⚠ 진단결과: 고장 데이터입니다! z축 폴리벨트 장력을 확인하십시오.

파일: 260-2.txt

- 평균 재구성 오류: 12345678.987654

✅ 진단결과: 정상 데이터입니다.

3. 주요 스크립트 사용법

◆ 학습 데이터 확인 `check_learning_data.py`

- `wafer_data/` 내 정상 상태 데이터 파일을 불러오고 각 파일별 데이터 크기를 확인 및 검증하는 용도
- Autoencoder 학습 전에 데이터 포맷과 크기를 사전 점검하기 위해 사용함

◆ 실행 방법 (로컬 기준) ← Docker 사용 안할시

```
cd ~/docker_autoenc/src  
python3 check_learning_data.py
```

◆ 실행 방법 (Docker 기준)

- `wafer_data`가 외부(로컬)에서 Docker내로 마운트 되는 경우

```
sudo docker run --rm -it ₩  
-v ~/ros2_ws/docker_autoenc/src/wafer_data:/workspace/src/wafer_data ₩  
autoenc-docker ₩  
python /workspace/src/check_learning_data.py
```


3. 주요 스크립트 사용법

◆ 학습 데이터 확인 `check_learning_data.py`

- `wafer_data/` 내 정상 상태 데이터 파일을 불러오고 각 파일별 데이터 크기를 확인 및 검증하는 용도
- Autoencoder 학습 전에 데이터 포맷과 크기를 사전 점검하기 위해 사용함

◆ 코드 동작 방식

단계	설명
1. 데이터 경로 설정	<code>./wafer_data</code> 폴더 내 데이터를 대상으로 동작
2. 분석 대상 파일 리스트	데이터 예시: <code>340hz50per.txt</code> , <code>340hz70per.txt</code> , <code>340hz100per.txt</code>
3. 데이터 로드 및 전처리	각 파일을 읽고, 필요한 열(Roll, Pitch, Yaw, <code>acc_x</code> , <code>acc_y</code> , <code>acc_z</code>)를 추출하여 <code>numpy</code> 배열로 저장
4. 데이터 형식 오류 필터링	잘못된 형식의 라인은 자동 무시(에러 출력 후 <code>continue</code>)
5. 파일 별 데이터 크기 저장	파일별 데이터 <code>shape</code> (샘플 수, 6개 컬럼)을 저장
6. 전체 데이터 합산 및 크기 출력	전체 데이터를 합산하여 최종 크기 출력

3. 주요 스크립트 사용법

◆ 학습 데이터 확인 `check_learning_data.py`

- `wafer_data/` 내 정상 상태 데이터 파일을 불러오고 각 파일별 데이터 크기를 확인 및 검증하는 용도
- Autoencoder 학습 전에 데이터 포맷과 크기를 사전 점검하기 위해 사용함

◆ 출력 예시

===== 개별 파일 데이터 크기 확인 =====

340hz50per.txt: (1000, 6)

340hz70per.txt: (1000, 6)

340hz100per.txt: (1000, 6)

350hz50per.txt: (1000, 6)

350hz70per.txt: (1000, 6)

350hz100per.txt: (1000, 6)

===== 전체 데이터 크기 =====

Total Data Size: (6000, 6)

3. 주요 스크립트 사용법

◆ 모델 학습 train_autoenc.py

- wafer_data/ 내 정상 상태 데이터 파일을 기반으로 Autoencoder를 학습하고 학습된 모델을 .pth 파일로 저장하며, 학습 Loss 시각화 및 결과를 파일로 출력

◆ 실행 방법 (로컬 기준) ← Docker 사용 안할시

```
cd ~/docker_autoenc/src  
python3 train_autoenc.py
```

◆ 실행 방법 (Docker 기준)

- wafer_data가 외부(로컬)에서 Docker내로 마운트 되는 경우

```
sudo docker run --rm -it ₩  
-v ~/docker_autoenc/src/wafer_data:/workspace/src/wafer_data ₩  
-v ~/docker_autoenc/models:/workspace/src/models ₩  
autoenc-docker ₩  
python /workspace/src/train_autoenc.py
```

3. 주요 스크립트 사용법

◆ 모델 학습 `train_autoenc.py`

- `wafer_data/` 내 정상 상태 데이터 파일을 기반으로 **Autoencoder**를 학습하고 학습된 모델을 `.pth` 파일로 저장하며, 학습 Loss 시각화 및 결과를 파일로 출력

◆ 코드 동작 방식

단계	설명
1. 데이터 로드 및 전처리	<code>NORMAL_FILES</code> 목록 기준으로 <code>wafer_data</code> 를 불러와 <code>roll</code> , <code>pitch</code> , <code>yaw</code> , <code>acc_x</code> , <code>acc_y</code> , <code>acc_z</code> 를 추출
2. 데이터 정규화	Z-score Normalization(평균 0, 표준 편차 1로 정규화)
3. DataLoader 구성	<code>TensorDataset</code> 및 <code>DataLoader</code> 로 배치 단위 학습 준비
4. Autoencoder 모델 정의	Encoder-Decoder 구조, <code>BatchNorm</code> , <code>LeakyReLU</code> 적용
5. 손실 함수 및 최적화	<code>L1Loss</code> , <code>AdamW Optimizer</code> (<code>weight_decay</code> 적용)
6. EarlyStopping 적용	개선 되지 않을 경우 조기 종료 (<code>patience = 10</code>)

3. 주요 스크립트 사용법

◆ 모델 학습 `train_autoenc.py`

- `wafer_data/` 내 정상 상태 데이터 파일을 기반으로 **Autoencoder**를 학습하고 학습된 모델을 `.pth` 파일로 저장하며, 학습 Loss 시각화 및 결과를 파일로 출력

◆ 코드 동작 방식

단계	설명
7. 학습 루프	에포크 별 Loss 계산 및 출력
8. 모델 저장	<code>/workspace/src/autoencoder_test.pth</code> 경로에 저장
9. 재구성 오류 평가	학습 데이터에 대한 최종 Reconstruction Loss 계산
10. Loss 그래프 저장	<code>tranining_loss.png</code> 파일로 저장 및 시각화

3. 주요 스크립트 사용법

◆ 모델 학습 `train_autoenc.py`

- `wafer_data/` 내 정상 상태 데이터 파일을 기반으로 **Autoencoder**를 학습하고 학습된 모델을 `.pth` 파일로 저장하며, 학습 Loss 시각화 및 결과를 파일로 출력

◆ 출력 예시

데이터 로드 완료! 데이터 크기: (4000, 6)

Epoch [1/100], Loss: 0.5234

Epoch [2/100], Loss: 0.4211

...

Epoch [15/100], Loss: 0.0217

Early Stopping Triggered at Epoch 15

Training Complete and Model Saved at `/workspace/src/autoencoder_test.pth`

Reconstruction Loss on Training Data: 0.0184

- `Training_loss.png` 파일이 `/workspace/src/` 경로에 생성됨

3. 주요 스크립트 사용법

◆ 고장 데이터 판별 2 `check_faulty_data2.py`

- 정상 데이터 기반으로 재구성 오류의 임계값(Threshold)를 자동 산출하고, 이 임계값을 기준으로 `wafer_data` 내 고장 데이터를 판별함
- 기존 `check_faulty_data.py` 와 달리 **k-sigma(3σ 등)** 기반 동적 임계값 계산을 수행

◆ 실행 방법 (로컬 기준) ← Docker 사용 안할시

```
cd ~/docker_autoenc/src  
python3 train_autoenc2.py
```

◆ 실행 방법 (Docker 기준)

- `wafer_data`가 외부(로컬)에서 Docker내로 마운트 되는 경우

```
sudo docker run --rm -it ₩  
-v ~/docker_autoenc/src/wafer_data:/workspace/src/wafer_data ₩  
-v ~/docker_autoenc/models:/workspace/src/models ₩  
autoenc-docker ₩  
python /workspace/src/check_faulty_data2.py
```

3. 주요 스크립트 사용법

◆ 모델 학습 train_autoenc.py

- wafer_data/ 내 정상 상태 데이터 파일을 기반으로 Autoencoder를 학습하고 학습된 모델을 .pth 파일로 저장하며, 학습 Loss 시각화 및 결과를 파일로 출력

◆ 코드 동작 방식

단계	설명
1. 모델 로드	models/autoencoder.pth 파일을 불러와 Autoencoder 네트워크에 로드
2. 정상 데이터 로딩	NORMAL_FILES 목록의 정상 데이터를 로드 및 전처리
3. 임계값 계산(k-sigma)	재구성 오류들의 평균과 표준편차를 이용해 임계값 = 평균 + k * 표준편차로 산출 (기본 k = 3)
4. 고장 데이터 판별	FAULTY_FILES 목록의 데이터를 재구성하고, 오류를 기준으로 고장 여부 판별
5. 고장 비율 기준 판정	파일별로 50% 이상 고장 데이터 비율 시 고장으로 판정

3. 주요 스크립트 사용법

◆ 모델 학습 train_autoenc.py

- wafer_data/ 내 정상 상태 데이터 파일을 기반으로 Autoencoder를 학습하고 학습된 모델을 .pth 파일로 저장하며, 학습 Loss 시각화 및 결과를 파일로 출력

◆ 코드 동작 방식

단계	설명
1. 모델 로드	models/autoencoder.pth 파일을 불러와 Autoencoder 네트워크에 로드
2. 정상 데이터 로딩	NORMAL_FILES 목록의 정상 데이터를 로드 및 전처리
3. 임계값 계산(k-sigma)	재구성 오류들의 평균과 표준편차를 이용해 임계값 = 평균 + k * 표준편차로 산출 (기본 k = 3)
4. 고장 데이터 판별	FAULTY_FILES 목록의 데이터를 재구성하고, 오류를 기준으로 고장 여부 판별
5. 고장 비율 기준 판정	파일별로 50% 이상 고장 데이터 비율 시 고장으로 판정

3. 주요 스크립트 사용법

◆ 모델 학습 `train_autoenc.py`

- `wafer_data/` 내 정상 상태 데이터 파일을 기반으로 **Autoencoder**를 학습하고 학습된 모델을 `.pth` 파일로 저장하며, 학습 Loss 시각화 및 결과를 파일로 출력

◆ 주요 차이점(vs `check_faulty_data.py`)

항목	<code>check_faulty_data.py</code>	<code>check_faulty_data2.py</code>
1. 임계값 설정	코드에 고정값(ex: 56250000)	정상 데이터 기반 k-sigma 동적 계산
2. 판별 기준	평균 오류 기반 판정	데이터별 재구성 오류 비율 기준 판정
3. 고장 비율 기준	없음(평균 오류 > 임계값)	고장 비율 50% 초과시 고장으로 판단

3. 주요 스크립트 사용법

◆ 모델 학습 train_autoenc.py

- wafer_data/ 내 정상 상태 데이터 파일을 기반으로 Autoencoder를 학습하고 학습된 모델을 .pth 파일로 저장하며, 학습 Loss 시각화 및 결과를 파일로 출력

◆ 출력 예시

[ 임계값 설정] 평균: 112345.67, 표준편차: 23456.78, 임계값 (k=3): 182716.01

파일: 260hz50per.txt

- 고장 데이터 비율: 95.40%
- 평균 재구성 오류: 523456.123456

 경고: 260hz50per.txt은 고장 데이터입니다!

파일: 240.txt

- 고장 데이터 비율: 10.50%
- 평균 재구성 오류: 92345.678901

 정상: 240.txt은 고장이 아닙니다!

4. 데이터 파일 규격(wafer_data/)

- ◆ 형식: CSV/ TXT(구분자: 콤마 ,)
- ◆ 분석 컬럼:

Roll	Pitch	Yaw	AccX	AccY	AccZ
------	-------	-----	------	------	------

- 필요 없는 열(ID, 시간 등)은 자동으로 제거됨

5. 사용시 참고사항

◆ 데이터 처리 관련

- `wafer_data/` 폴더 내 데이터 파일들은 코드 내 지정된 목록(NORMAL_FILES, FAULTY_FILES)를 기준으로 불러옴
- 신규 파일 추가 시 코드 내 리스트를 직접 수정해야 분석 대상에 포함됨
- 예: `NORMAL_FILES`, `FAULTY_FILES` 목록 추가

◆ 판정 기준(임계값 설정:THRESHOLD)

- 기본 판정 기준은 재구성 오류(Mean Squared Error, L1Loss 등)을 기반으로 동작
- `check_faulty_data.py` : 고정 임계값(THRESHOLD)를 코드내 수치로 지정
- `check_faulty_data2.py` : 정상 데이터를 이용한 k-sigma 방식(3σ 등) 으로 동적 임계값 계산
- 판정 기준 변경이 필요하다면: `THRESHOLD`, `THRESHOLD_K` 값 변경 가능
- 고장 판별 기준(고장 비율 50% 등)도 코드 내에서 수정 가능

5. 사용시 참고사항

◆ 모델 재학습 필수 상황

- Wafer_data 정상 데이터에 변경/추가가 발생하면 반드시 `train_autoenc.py`로 **Autoencoder**를 재학습해야 함
- 기존 모델(`autoencoder.pth`)은 이전 데이터 기준이므로 최신 데이터 반영 필요
- 재학습 후 결과 모델 파일(.pth)는 `check_faulty_data.py`, `check_faulty_data2.py` 에서 활용됨

◆ 결과 출력 및 로그 활용

- 현재 결과는 **터미널 표준 출력(stdout)** 형태로 제공
 - 각 파일별 평균 재구성 오류, 고장 비율, 최종 판별 결과 등
- 확장 시, **결과를 로그 파일(.txt)로 저장하거나 CSV 파일로 결과 리포팅**하는 것도 가능(필요시 코드 수정)
 - 예: 판별 결과물 `/workspace/src/results/` 폴더에 저장

5. 사용시 참고사항

◆ GPU/CPU 환경 지원

- 코드 실행 시 **GPU 가용 여부를 인식하여** CUDA 또는 CPU로 동작
 - GPU 없는 환경에서도 동작하지만 처리 속도는 차이가 발생할 수 있음

◆ 시각화 결과물

- 모델 학습 시, **학습 Loss 그래프(training_loss.png)**가 자동 저장 됨
 - 경로: `/workspace/src/training_loss.png`
 - 학습 경향을 시각적으로 확인 가능